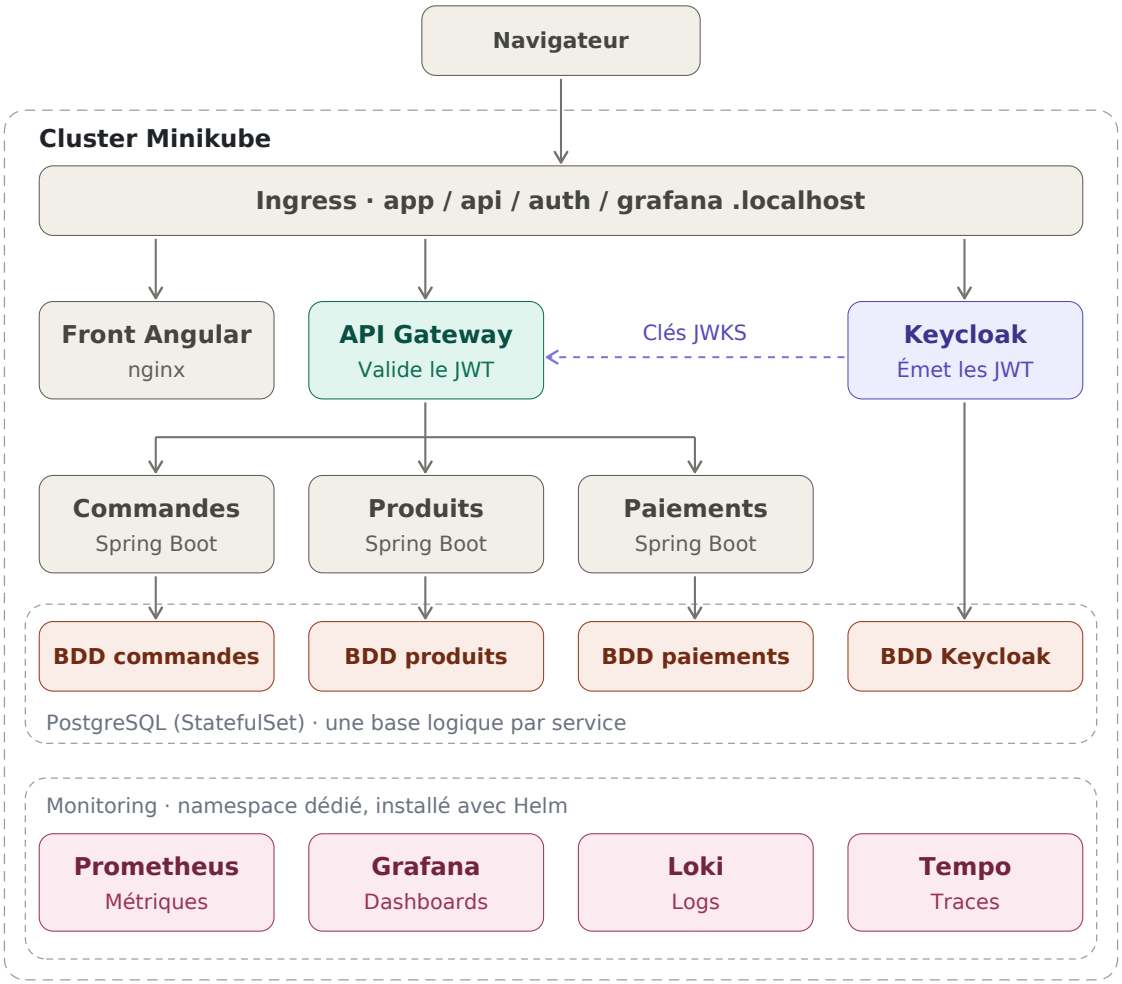
Architecture et arborescence du projet

Architecture microservices avec service d'authentification — Angular · Spring Boot · Keycloak · PostgreSQL · Docker · Minikube · Prometheus/Grafana · SonarQube

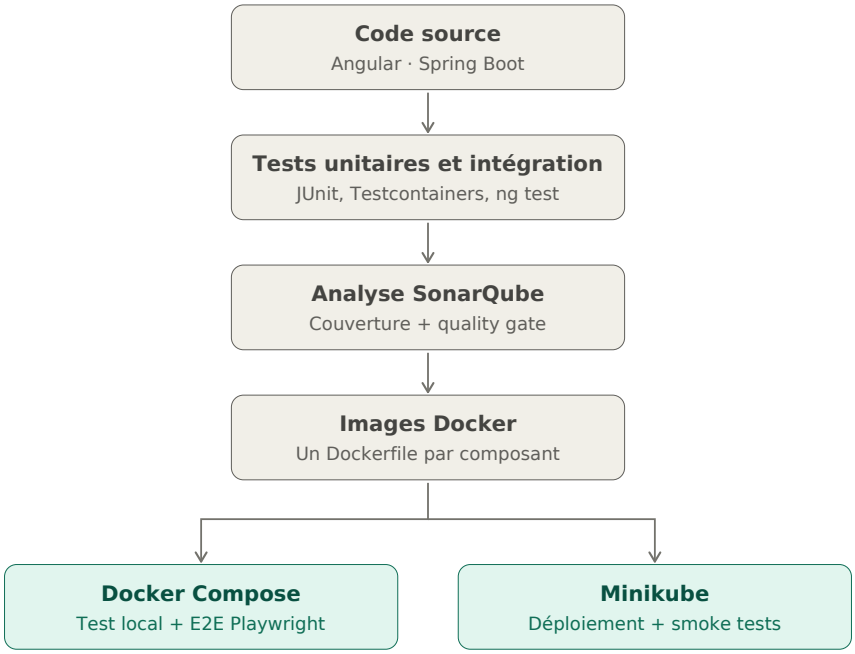
Architecture de l'application

L'application telle qu'elle tourne dans Minikube. Le navigateur passe par l'Ingress. La gateway valide les JWT émis par Keycloak grâce à ses clés publiques (JWKS), puis route vers les microservices. Chaque service a sa base logique dans un PostgreSQL commun. Prometheus lit /actuator/prometheus sur chaque composant, les traces partent vers Tempo et les logs vers Loki.



Cycle de développement et de déploiement

SonarQube et Docker Compose interviennent avant le déploiement : le code est testé, analysé, puis construit en images. Les mêmes images sont ensuite lancées en Docker Compose pour les tests E2E, ou dans Minikube.



Vue d'ensemble

Le projet est un monorepo. Chaque dossier de premier niveau a un rôle précis : le code, la configuration partagée, les environnements d'exécution, les tests et l'automatisation.

```
projet/
├── README.md                    # présentation, prérequis, lancement
├── .gitignore
├── .editorconfig                # règles de formatage communes
├── .env.example                 # variables docker-compose (copier en .env)
├── docs/                        # documentation, rapport, décisions techniques
├── frontend/                    # application Angular
├── gateway/                     # Spring Cloud Gateway (point d'entrée de l'API)
├── services/                    # microservices Spring Boot
│   ├── produits/
│   ├── commandes/
│   └── paiements/
├── keycloak/                    # realm Keycloak versionné
├── database/                    # script d'initialisation PostgreSQL
├── docker/                      # environnement de test (Docker Compose)
├── k8s/                         # environnement cible (Minikube + Helm)
├── tests/                       # tests sur l'application déployée
└── scripts/                     # automatisation (build, déploiement, tests)
```

Rôle	Dossiers
Code (avec Dockerfile)	frontend/, gateway/, services/
Configuration partagée	keycloak/, database/
Environnements	docker/ (test), k8s/ (Minikube)
Qualité	src/test de chaque composant, tests/, SonarQube
Automatisation et documentation	scripts/, docs/

1. Documentation — docs/

Sert directement pour le rapport et la soutenance. Les décisions techniques sont tracées sous forme d'ADR (Architecture Decision Records).

```
docs/
├── architecture.md                  # schéma et explication de l'architecture
├── authentification.md              # flux OIDC, rôles, clients Keycloak
├── deploiement.md                   # procédure Docker puis Minikube
├── monitoring.md                    # dashboards, alertes, scénario de démo
├── tests.md                         # stratégie de tests
├── decisions/                       # choix techniques justifiés (ADR)
│   ├── 001-keycloak.md
│   ├── 002-database-per-service.md
│   └── 003-minikube.md
└── images/                          # schémas exportés pour le rapport
```

2. Front-end — frontend/

Application Angular. Le dossier **core/** contient ce qui est transverse (configuration, authentification, intercepteurs), **shared/** ce qui est réutilisable, et **features/** un dossier par domaine métier. Les tests unitaires (*.spec.ts) sont placés à côté du code testé.

```
frontend/
├── Dockerfile                       # multi-stage : node (ng build) → nginx
├── .dockerignore
├── nginx.conf                       # SPA fallback, cache, gzip
├── package.json
├── package-lock.json
├── angular.json
├── tsconfig.json
├── tsconfig.app.json
├── tsconfig.spec.json
├── sonar-project.properties         # config SonarQube du front
└── src/
    ├── index.html
    ├── main.ts                      # charge config.json avant le démarrage
    ├── styles.scss
    ├── assets/
    │   └── config.json              # URLs (auth, API), valeurs locales par défaut
    └── app/
        ├── app.component.ts
        ├── app.config.ts            # providers : OIDC, HTTP, intercepteurs
        ├── app.routes.ts            # routes + guards
        ├── core/                    # chargé une fois, transverse
        │   ├── config/
        │   │   ├── app-config.service.ts        # lit config.json
        │   │   └── app-config.service.spec.ts
        │   ├── auth/
        │   │   ├── auth.config.ts               # authority, clientId, PKCE
        │   │   ├── auth.service.ts              # login, logout, user, rôles
        │   │   ├── auth.service.spec.ts
        │   │   ├── auth.guard.ts                # utilisateur connecté ?
```

```
│   │   ├── auth.guard.spec.ts
│   │   ├── role.guard.ts                # a le rôle requis ?
│   │   └── role.guard.spec.ts
│   ├── interceptors/
│   │   ├── auth.interceptor.ts          # ajoute le Bearer vers l'API
│   │   ├── auth.interceptor.spec.ts
│   │   ├── error.interceptor.ts         # gestion 401 / 403 / 500
│   │   └── error.interceptor.spec.ts
│   └── layout/
│       ├── header/                      # menu, bouton login/logout
│       └── footer/
├── shared/                    # réutilisable partout
│   ├── components/            # boutons, tableaux, loaders...
│   ├── models/                # interfaces TypeScript (Produit, Commande...)
│   └── pipes/
└── features/                  # un dossier par domaine métier
    ├── accueil/
    ├── produits/
    │   ├── produits.routes.ts
    │   ├── services/
    │   │   ├── produit-api.service.ts
    │   │   └── produit-api.service.spec.ts
    │   ├── liste-produits/
    │   │   ├── liste-produits.component.ts
    │   │   ├── liste-produits.component.html
    │   │   ├── liste-produits.component.scss
    │   │   └── liste-produits.component.spec.ts
    │   └── detail-produit/
    ├── commandes/             # même organisation
    ├── paiements/             # même organisation
    └── admin/                 # écrans réservés au rôle admin
```

3. Gateway — gateway/

Point d'entrée unique de l'API. Elle valide le JWT, applique le CORS et route les requêtes vers les microservices. Elle n'a pas de base de données. Les profils Spring ne contiennent que ce qui change d'un environnement à l'autre.

```
gateway/
├── Dockerfile                         # multi-stage : maven → JRE
├── .dockerignore
├── pom.xml                            # + JaCoCo, Failsafe, Sonar
└── src/
    ├── main/
    │   ├── java/com/projet/gateway/
    │   │   ├── GatewayApplication.java
    │   │   └── config/
    │   │       ├── SecurityConfig.java          # validation JWT
    │   │       ├── JwtConfig.java               # JWKS interne + issuer attendu
    │   │       └── CorsConfig.java
    │   └── resources/
    │       ├── application.yml                  # routes, actuator, tracing
    │       ├── application-docker.yml
    │       └── application-k8s.yml
    └── test/
        ├── java/com/projet/gateway/
        │   ├── config/SecurityConfigTest.java   # 401 sans token, routes protégées
        │   └── integration/RoutesIT.java        # routage vers WireMock
        └── resources/
            └── application-test.yml
```

4. Microservices — services/

Chaque service est indépendant : son propre Dockerfile, sa propre base (database per service) et son schéma géré par Flyway. Le service **produits** est détaillé ci-dessous ; **commandes** et **paiements** suivent la même structure.

```
services/produits/
├── Dockerfile
├── .dockerignore
├── pom.xml                          # web, security, jpa, flyway, actuator,
│                                    # micrometer, otel, testcontainers, jacoco
└── src/
    ├── main/
    │   ├── java/com/projet/produits/
    │   │   ├── ProduitsApplication.java
    │   │   ├── config/
    │   │   │   ├── SecurityConfig.java          # règles d'accès par rôle
    │   │   │   ├── KeycloakRoleConverter.java   # realm_access.roles → ROLE_*
    │   │   │   └── JwtConfig.java               # JWKS interne + issuer attendu
    │   │   ├── controller/
    │   │   │   └── ProduitController.java
    │   │   ├── service/
    │   │   │   └── ProduitService.java
    │   │   ├── repository/
    │   │   │   └── ProduitRepository.java
    │   │   ├── model/
    │   │   │   └── Produit.java                 # entité JPA
    │   │   ├── dto/
    │   │   │   ├── ProduitRequest.java
    │   │   │   └── ProduitResponse.java
    │   │   ├── mapper/
    │   │   │   └── ProduitMapper.java
    │   │   ├── exception/
    │   │   │   ├── ProduitNotFoundException.java
    │   │   │   └── GlobalExceptionHandler.java  # erreurs HTTP propres
    │   │   └── metrics/
    │   │       └── ProduitMetrics.java          # métriques métier Micrometer
    │   └── resources/
    │       ├── application.yml                  # BDD, JWT, actuator, tracing, logs JSON
    │       ├── application-docker.yml
    │       ├── application-k8s.yml
    │       └── db/migration/                    # Flyway
    │           ├── V1__create_table_produit.sql
    │           └── V2__insert_donnees_initiales.sql
    └── test/
        ├── java/com/projet/produits/
        │   ├── unit/
        │   │   ├── ProduitServiceTest.java      # Mockito
        │   │   └── ProduitMapperTest.java
        │   ├── web/
        │   │   └── ProduitControllerTest.java   # @WebMvcTest + jwt() : 401/403/200
        │   ├── repository/
        │   │   └── ProduitRepositoryTest.java   # @DataJpaTest + Testcontainers
        │   ├── integration/
        │   │   └── ProduitApiIT.java            # @SpringBootTest + PostgreSQL réel
        │   └── contract/
        │       └── ProduitProviderPactIT.java   # bonus : vérifie le contrat
        └── resources/
            └── application-test.yml
```

Le service **commandes** reprend cette structure et ajoute l'appel au service produits (avec un token obtenu en client credentials) :

```
services/commandes/                  # même structure que produits, avec en plus :
└── src/
    ├── main/java/com/projet/commandes/
    │   ├── client/
    │   │   └── ProduitClient.java               # appel à l'API produits
    │   └── config/
    │       └── OAuth2ClientConfig.java          # token client credentials
    └── test/java/com/projet/commandes/
        ├── integration/
        │   └── CommandeApiIT.java               # produits simulé par WireMock
        └── contract/
            └── ProduitConsumerPactTest.java     # bonus : définit le contrat attendu
```

```
services/paiements/                    # même structure que produits
```

5. Configuration partagée — keycloak/ et database/

Utilisée à la fois par Docker Compose et par Kubernetes. Keycloak et PostgreSQL utilisent les images officielles : pas de Dockerfile. Le script d'initialisation ne s'exécute qu'au premier démarrage, quand le volume est vide.

```
keycloak/
├── realm-export.json                  # realm "projet" : clients, rôles, scopes
├── realm-test.json                    # users de test (mots de passe connus) pour E2E
└── themes/                            # optionnel : page de login personnalisée
    └── projet/login/

database/
└── init/
    └── 01-create-databases.sql        # keycloak_db, produits_db, commandes_db,
                                       # paiements_db + un utilisateur par base
```

6. Environnement de test — docker/

Pour tester rapidement l'application complète. Le monitoring et SonarQube sont dans des fichiers séparés, ajoutés seulement quand on en a besoin.

```
docker/
├── docker-compose.yml                 # postgres, keycloak, services, gateway, front
├── docker-compose.monitoring.yml      # grafana/otel-lgtm (optionnel)
├── docker-compose.sonar.yml           # SonarQube + sa base (outillage, à part)
└── config/
    └── frontend-config.json           # config.json du front pour Docker
```

7. Environnement cible — k8s/ (Minikube)

L'application est déployée avec des manifests YAML écrits à la main. Le monitoring est installé avec Helm : le dossier **monitoring/** ne contient que les fichiers values qui personnalisent les charts, plus les ServiceMonitor, les alertes et les dashboards.

```
k8s/
├── namespace.yaml                     # namespace "projet"
├── ingress.yaml                       # app / api / auth .localhost
├── database/
│   ├── postgres-secret.yaml
│   ├── postgres-init-configmap.yaml   # contenu de 01-create-databases.sql
│   ├── postgres-pvc.yaml
│   ├── postgres-statefulset.yaml
│   └── postgres-service.yaml
├── keycloak/
│   ├── keycloak-secret.yaml           # admin + accès BDD
│   ├── keycloak-realm-configmap.yaml  # contenu de realm-export.json
│   ├── keycloak-deployment.yaml       # KC_HOSTNAME, métriques, probes
│   └── keycloak-service.yaml
├── gateway/
│   ├── gateway-configmap.yaml
│   ├── gateway-deployment.yaml        # probes actuator, resources
│   └── gateway-service.yaml
├── services/
│   ├── produits/
│   │   ├── produits-configmap.yaml    # URL BDD, issuer, JWKS, OTLP
│   │   ├── produits-secret.yaml       # mot de passe BDD
│   │   ├── produits-deployment.yaml
│   │   └── produits-service.yaml
│   ├── commandes/                     # même structure + secret client credentials
│   └── paiements/                     # même structure
├── frontend/
```

```
│   ├── frontend-configmap.yaml        # config.json pour Minikube
│   ├── frontend-deployment.yaml
│   └── frontend-service.yaml
└── monitoring/                        # installé avec Helm
    ├── values-kube-prometheus-stack.yaml   # Prometheus, Grafana, Alertmanager
    ├── values-loki.yaml                    # optionnel : stockage des logs
    ├── values-alloy.yaml                   # optionnel : collecte des logs
    ├── values-tempo.yaml                   # optionnel : traces
    ├── servicemonitors.yaml           # scrape /actuator/prometheus + Keycloak
    ├── prometheus-rules.yaml          # alertes : service down, 5xx, latence
    ├── grafana-ingress.yaml           # grafana.localhost
    └── dashboards/
        ├── dashboards-configmap.yaml  # chargés automatiquement par Grafana
        ├── microservices-overview.json     # req/s, latence p95, erreurs
        ├── jvm.json
        └── keycloak.json              # logins réussis / échoués
```

8. Tests sur l'application déployée — tests/

Les tests unitaires et d'intégration vivent dans chaque composant (src/test). Ce dossier contient les tests qui s'exécutent contre l'application lancée en Docker ou dans Minikube.

```
tests/
├── e2e/                               # Playwright, sur l'environnement Docker
│   ├── package.json
│   ├── playwright.config.ts
│   ├── fixtures/
│   │   └── auth.fixture.ts            # login Keycloak réutilisable
│   └── specs/
│       ├── login.spec.ts
│       ├── produits.spec.ts
│       ├── commandes.spec.ts
│       └── droits-admin.spec.ts       # user simple bloqué sur l'admin
├── functional/                        # optionnel : BDD Cucumber
│   └── features/
│       ├── passer-commande.feature
│       └── gestion-produits.feature
├── contract/                          # optionnel : contrats Pact générés
│   └── pacts/
├── performance/                       # k6
│   ├── charge-produits.js
│   └── charge-commandes.js
└── smoke/
    └── smoke-test.sh                  # health, token, API via Ingress
```

9. Automatisation — scripts/

Les scripts évitent de retaper les commandes et documentent l'ordre de déploiement. Ils permettent aussi au correcteur de lancer le projet facilement.

```
scripts/
├── build-images.sh                    # docker build de front, gateway, services
├── run-all-tests.sh                   # mvn verify + ng test
├── sonar-analyse.sh                   # analyse de tous les composants + quality gate
├── docker-up.sh                       # lance l'env Docker (+ monitoring si option)
├── run-e2e.sh                         # docker up + Playwright
├── minikube-setup.sh                  # minikube start + addons ingress/metrics
├── load-images.sh                     # minikube image load
├── deploy-app.sh                      # BDD → Keycloak → services → front → ingress
├── deploy-monitoring.sh               # helm repo add + helm install + servicemonitors
├── smoke-test-k8s.sh
└── teardown.sh                        # nettoyage Docker et Minikube
```

10. Repères et enchaînement

- **Un Dockerfile par composant codé** (front, gateway, chaque service). Keycloak, PostgreSQL, SonarQube et la stack de monitoring utilisent des images officielles.
- **docker/ et k8s/ déploient les mêmes images.** On les construit une fois, puis on lance soit Docker Compose, soit Minikube.
- **Les profils Spring** (application-docker.yml, application-k8s.yml) ne contiennent que ce qui change. Les valeurs sensibles arrivent par variables d'environnement (.env ou Secrets Kubernetes).
- **Le schéma de chaque base** appartient à son service (Flyway). Le dossier database/ ne fait que créer les bases et les utilisateurs.
- **Piège de l'issuer** : les services récupèrent les clés via l'URL interne de Keycloak (JWKS) mais vérifient l'issuer public vu par le navigateur.

Ordre d'exécution global :

```
1. mvn verify / ng test            → unitaires + intégration + couverture
2. scripts/sonar-analyse.sh        → qualité du code + quality gate
3. scripts/build-images.sh         → images Docker
4. docker compose up + tests/e2e   → tests fonctionnels sur l'application complète
5. déploiement Minikube            → puis tests/smoke
6. k6 (optionnel)                  → charge + observation dans Grafana
```